

UC LINA 600 LINEAR ALGEBRA FOR GRAPHICS · MODULE 06 · LECTURE NOTES

Cross Products and Normals

Perpendicularity, area, and the handedness trap.

MODULE OUTCOMES

- Compute cross products and interpret them.
- Compute face normals from a triangle.
- Explain handedness and winding order.
- Explain why normals transform by the inverse transpose.
- Debug a normal that points the wrong way.

1 What it gives you

THE CROSS PRODUCT RETURNS A VECTOR PERPENDICULAR TO BOTH, WITH LENGTH EQUAL TO THE PARALLELOGRAM'S AREA

Direction: perpendicular to both inputs — which is exactly what you want for a surface normal, and is the use that dominates in graphics.

Magnitude: $|a||b| \sin \theta$ — the area of the parallelogram the two vectors span, and **half of that is the triangle's area**, which is how mesh surface areas and barycentric weights get computed without any trigonometry.

And it is the zero vector exactly when the inputs are parallel — which is the **degenerate-triangle test** (Module 02 §2's linear dependence) and **is worth checking before you normalise** (§2).

Plus it is anticommutative: $a \times b = -(b \times a)$ — which is where handedness enters and is §3's subject. Note it is also not associative, so parenthesise.

2 Face normals

GIVEN a triangle with vertices p0, p1, p2:

```
e1 = p1 - p0
e2 = p2 - p0
n  = cross(e1, e2)
area = 0.5 * length(n)
n  = normalize(n)
```

TWO THINGS GO WRONG

1 DEGENERATE TRIANGLE

if p0, p1, p2 are collinear then n is the zero vector and normalize() divides by zero. CHECK length(n) before normalising -- this is not paranoia, real meshes contain these, and the resulting NaN propagates silently into every lighting calculation downstream.

2 WINDING ORDER

cross(e1, e2) and cross(e2, e1) point opposite ways. Which one counts as "outward" depends on the order the vertices are listed in, which is a mesh convention. Counter-clockwise-is-front is the usual choice. PICK ONE AND WRITE IT DOWN.

AND VERTEX NORMALS are the normalised average of the face normals meeting at that vertex.

Check the length before normalising — real meshes contain degenerate triangles (exported from CAD, produced by decimation, or simply authored badly) **and the NaN propagates silently through everything downstream**, producing black or invisible geometry far from the actual fault. **Inconsistent winding is the usual cause of a mesh that shades in patches** — some faces lit and their neighbours dark — because the averaged vertex normals partially cancel.

3 Handedness

THE CROSS PRODUCT'S DIRECTION IS A CONVENTION, AND MIXING CONVENTIONS IS THE COMMONEST BUG IN GRAPHICS

Right-handed: point the fingers of your right hand along $\mathbf{a}$, curl them toward $\mathbf{b}$, and your thumb gives the direction of $\mathbf{a} \times \mathbf{b}$. **Left-handed systems reverse it** — and both are entirely self-consistent, which is why neither can be called wrong.

And the conventions genuinely differ in practice: OpenGL and essentially all mathematics are right-handed; DirectX and several engines are left-handed — which is a historical accident that you inherit and cannot avoid.

So the bug is mixing them — importing a mesh authored under one convention into a renderer that assumes the other, or calling a library function that disagrees with your own code — and the symptom is normals inverted, faces culled backwards, or the whole scene mirrored.

Which is why the rule is: choose once, write it down somewhere visible, and check it at every boundary — every import, every export, and every library call — because nothing will warn you and the numbers are valid either way.

4 Transforming normals

$$\text{need } \mathbf{n}' \cdot (\mathbf{M}\mathbf{v}) = 0 \text{ whenever } \mathbf{n} \cdot \mathbf{v} = 0 \Rightarrow \mathbf{n}' = (\mathbf{M}^{-1})^T \mathbf{n}$$

- **A normal is not an independent direction:** it is defined by being perpendicular to the surface, which is a condition relating it to the tangent vectors rather than a property it carries on its own.
- **So after the tangent vectors are transformed by $\mathbf{M}$, the perpendicularity must survive** — we need the transformed normal to be perpendicular to the transformed tangents — **and writing that condition out forces $\mathbf{n}' = (\mathbf{M}^{-1})^T \mathbf{n}$** , with no freedom in the matter.
- **And for a pure rotation, $(\mathbf{M}^{-1})^T = \mathbf{M}$** (since $\mathbf{M}^{-1} = \mathbf{M}^T$ for an orthogonal matrix, Module 04 §2) — **so nothing changes, which is why the issue only appears under non-uniform scale and why it surprises people who have only ever rotated things.**
- **Under non-uniform scale, transforming a normal by $\mathbf{M}$ tilts it off the surface** — squash a sphere into an ellipsoid and the naively-transformed normals no longer point away from it — **and the inverse transpose is exactly what keeps it perpendicular. This is Module 01 §2's surprising row, and the explanation is geometric rather than algebraic.**

AND THE DEBUGGING PROCEDURE FOR A WRONG NORMAL

Draw it. Render the normals as short line segments from each vertex or face centre — which turns an invisible bug into an obvious one, and takes about ten minutes to implement and pays for itself immediately.

Then check, in this order: the winding order (§2), the handedness convention (§3), whether you used the inverse transpose (§4), and whether you normalised *after* transforming rather than before.

That last one is easy to miss: a transform can change a unit normal's length even when the direction is handled correctly, and the shading will then be subtly wrong rather than obviously broken — too bright or too dark in a way that reads as an art problem.

Which is four causes, and in this program's experience it is nearly always one of them — so the list is worth keeping next to the renderer.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
3Blue1Brown, chapters 10 and 11 (free)	§1 — the cross product, including the determinant connection that Module 09 develops.
Lengyel, chapters 1 and 4 (library copy)	§§2 to 4 — normals, winding, and the normal matrix, in graphics conventions.
Scratchapixel, the normals and shading articles (free)	§§2 and 4 — with the inverse transpose derived rather than asserted.
The OpenGL and DirectX coordinate system documentation (free)	§3 — the two conventions stated by their own specifications, which is the authoritative source when they disagree.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Compute five cross products** and verify perpendicularity by dot product.
- 2 **Verify the area interpretation** against a known triangle.
- 3 **Construct a degenerate triangle** and watch normalise produce NaN.
- 4 **Add the length check** and handle it.
- 5 **Reverse a triangle's winding** and observe the normal flip.

- 6 **State your handedness convention** and find where your tools disagree.
- 7 **Transform a normal by a non-uniform scale**, both ways, and compare.
- 8 **Derive the inverse transpose** from the perpendicularity condition.
- 9 **Implement normal visualisation** as line segments.
- 10 **Introduce each of the four bugs deliberately** and learn what each looks like.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What three things does a cross product give you?
- 2 When is it zero, and what does that mean for a mesh?
- 3 Give the face normal computation.
- 4 Name the two things that go wrong, and the fix for each.
- 5 What determines which way is outward?
- 6 State the handedness conventions and who uses which.
- 7 What does mixing them look like?
- 8 Why do normals use the inverse transpose?
- 9 When does it not matter, and why?
- 10 Give the four-item debugging list.